

🔒 Complemento de Infraestructura y Seguridad

Elementos documentados en la arquitectura que no fueron implementados en el despliegue

Mayo 2026

Docker +

SSL + ~~BaseAuthChecks~~

Diagrama_Arquitectura_LaEsperanza.html

1. SSL / HTTPS — No implementado

El **Diagrama de Arquitectura** documenta la capa de Gateway con "SSL / CORS — HTTPS + CORS". En la implementación actual:

ASPECTO	DOCUMENTADO	REALIDAD	
HTTPS	Termina SSL en producción	Nginx solo escucha en puerto 80 (HTTP)	✖
CORS	Backend acepta solo el dominio del frontend	<code>origin: process.env.FRONTEND_URL ?? '*'</code>	⚠ Parcial

Riesgo: La ausencia de HTTPS expone las comunicaciones a ataques Man-in-the-Middle (MITM), permitiendo la interceptación de tokens JWT y contraseñas.

Configuración Nginx recomendada con SSL

```

server {
    listen 443 ssl http2;
    server_name laesperanza.app;

    ssl_certificate      /etc/ssl/certs/laesperanza.crt;
    ssl_certificate_key  /etc/ssl/private/laesperanza.key;
    ssl_protocols        TLSv1.2 TLSv1.3;
    ssl_ciphers           HIGH:!aNULL:!MD5;

    root    /usr/share/nginx/html;
    index  index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location /api/ {
        proxy_pass          http://backend:3000;
        proxy_http_version  1.1;
        proxy_set_header    Upgrade $http_upgrade;
        proxy_set_header    Connection 'upgrade';
        proxy_set_header    Host $host;
        proxy_set_header    X-Real-IP $remote_addr;
        proxy_set_header    X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header    X-Forwarded-Proto $scheme;
        proxy_cache_bypass  $http_upgrade;
    }
}

server {
    listen 80;
    server_name laesperanza.app;
    return 301 https://$host$request_uri;
}

```

Recomendación: Usar Let's Encrypt + Certbot para certificados SSL gratuitos, o configurar un reverse proxy como Caddy que maneja SSL automáticamente.

🔗 2. Health Checks — Solo DB implementado

La arquitectura documenta health checks para todos los contenedores. Actualmente solo la base de datos tiene healthcheck configurado.

Health check faltante para el backend

```

backend:
  healthcheck:
    test: ["CMD", "wget", "--no-verbose", "--tries=1", "--spider", "http://localhost:3000/api/v1/"]
    interval: 30s
    timeout: 10s
    retries: 3
    start_period: 15s

```

Health check faltante para el frontend

```
frontend:
  healthcheck:
    test: ["CMD", "wget", "--no-verbose", "--tries=1", "--spider", "http://localhost:80/"]
    interval: 30s
    timeout: 10s
    retries: 3
    start_period: 15s
```

Recomendación: El healthcheck del backend ya tiene un endpoint `GET /api/v1/health` implementado. Solo falta agregar la configuración en `docker-compose.yml`. Para el frontend, verificar que Nginx responda 200.

♥ 3. Seguridad Adicional No Implementada

MEDIDA DE SEGURIDAD	DOCUMENTADA	IMPLEMENTADA	
Rate limiting en login	No explícita en docs de arquitectura pero es buena práctica	✗ Sin protección contra fuerza bruta	✗
Validación de entrada (sanitización)	Implícita en el diseño de API	⚠ Se valida existencia de campos pero no sanitización profunda (XSS, inyección)	⚠ Parcial
Helmet (headers de seguridad HTTP)	No documentada explícitamente	✗ No instalado ni configurado	✗
Variables de entorno en .env	✓ Documentado	✓ Implementado	✓
Contraseñas hashadas con bcrypt	✓ Documentado (salt=10)	✓ Implementado	✓
JWT con expiración configurable	✓ Documentado	✓ JWT_EXPIRES_IN configurable (default 7d)	✓

Recomendaciones de seguridad:

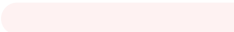
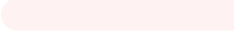
- Agregar `express-rate-limit` para limitar intentos de login a 5 por minuto
- Agregar `helmet` para headers de seguridad HTTP
- Agregar `express-validator` para sanitización de entradas
- Configurar correctamente `CORS` en producción con dominio específico
- Usar `cookie-http-only` para almacenar el token JWT en lugar de `localStorage`

4. Docker Compose — Diferencias detectadas

ASPECTO	DOCUMENTADO	IMPLEMENTADO	
Puerto DB externo	5432 (estándar PostgreSQL)	5433:5432 (puerto no estándar)	 Diferente
Volumen pgdata	✓ Documentado	✓ pgdata:/var/lib/postgresql/data	
named volume vs bind mount	Volumen Docker	Named volume pgdata	
dependencias entre servicios	Backend depende de DB, Frontend depende de Backend	Backend depende de DB (condition: service_healthy). Frontend depende solo de backend (sin health)	 Parcial

Nota: El puerto 5433 se usa probablemente para evitar conflicto con una instalación local de PostgreSQL en el puerto 5432. Es correcto mientras sea consistente con la variable DATABASE_URL en .env.

5. Resumen de brechas de infraestructura

ELEMENTO	PRIORIDAD	ESFUERZO ESTIMADO
SSL/HTTPS en Nginx	 Alta	2-4 horas (configuración + certificados)
Health checks en backend y frontend	 Media	30 minutos
Rate limiting en login	 Media	1 hora (instalar + configurar express-rate-limit)
Helmet (headers HTTP seguros)	 Media	30 minutos
Validación de entrada con express-validator	 Media	2-3 horas (agregar a todos los endpoints)
Migración de localStorage a cookies httpOnly	 Alta	4-6 horas (cambios en frontend + backend)